RETAIL INVENTORY & SALES MANAGEMENT

Design and Implementation of a Database Management System for Retail Inventory and Sales Management System



DBMS Project — Review 2

TEAM MEMBERS	Kashvi Sahoo 25WU0102118
COURSE	Database Management Systems
UNIVERSITY	Woxsen University

Problem Statement & Objectives

The Problem

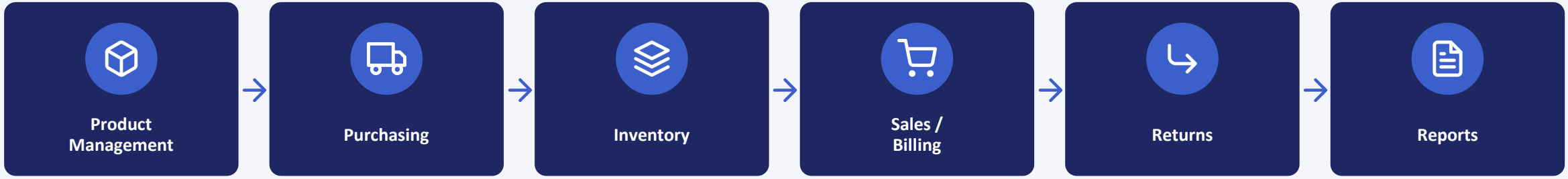
Retail stores manage products, suppliers, purchases, sales, customers, returns and stock. Manual or poorly structured systems often lead to:

- Incorrect stock levels
- Duplicate product / invoice records
- Inconsistent sales and purchase data
- Difficult tracking of returns and payments
- Time-consuming report generation

Project Objectives

- Design a normalized relational database up to 3NF
- Maintain accurate product and inventory records
- Manage purchases, sales, customers, suppliers and returns
- Enforce data integrity using SQL constraints
- Generate useful business reports using SQL queries and views

System Modules & Database Entities



Modules progress from product setup through purchasing, inventory tracking, billing, returns and reporting.

Major Database Entities

Connected through primary and foreign keys

Product	Category	Brand	Supplier	Purchase
Purchase Item	Customer	Sale	Sale Item	Return
Stock Movement	Invoice	Payment		

Normalization & Functional Dependencies

Unnormalized Data → 1NF → 2NF → 3NF

1NF

- Atomic values only
- No repeating groups
- Each field holds a single value

2NF

- Satisfies 1NF
- Removes partial dependencies
- Non-key attributes depend on the full primary key

3NF

- Satisfies 2NF
- Removes transitive dependencies
- Non-key attributes depend only on the primary key

Sample Functional Dependencies

ProductID → ProductName, Price, CategoryID, BrandID

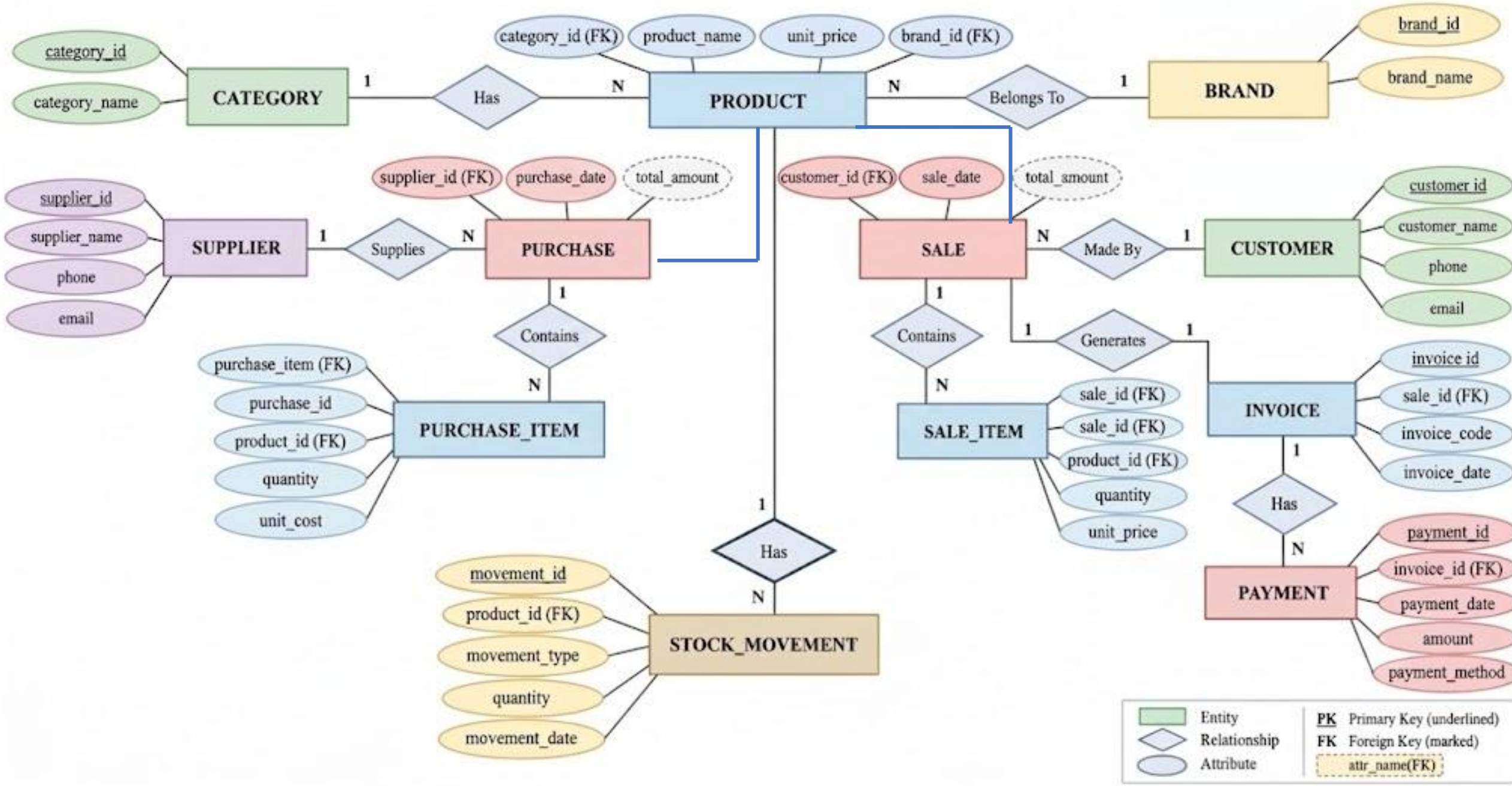
SupplierID → SupplierName, Contact, Email

CustomerID → CustomerName, Phone

InvoiceID → InvoiceDate, CustomerID, PaymentID

RETAIL INVENTORY & SALES MANAGEMENT SYSTEM

ER DIAGRAM



Database Implementation & Integrity Constraints

Implemented using MySQL

```
CREATE TABLE Product (
  ProductID INT PRIMARY KEY AUTO_INCREMENT,
  ProductCode VARCHAR(20) UNIQUE NOT NULL,
  ProductName VARCHAR(100) NOT NULL,
  Price DECIMAL(10,2) CHECK (Price > 0),
  StockQty INT DEFAULT 0
    CHECK (StockQty >= 0)
);
```

Implemented Constraints

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

Business Rules Enforced

- Product and invoice codes must be unique
- Price and quantity must be positive
- Stock cannot become negative
- Returned quantity cannot exceed sold quantity
- Payment amount cannot exceed the sale amount

SQL Queries & Reports

Joins

Combine Product, Category, Supplier, Sale and Customer data

Aggregation

SUM(), COUNT(), AVG(), GROUP BY

Subqueries

Identify products or suppliers based on calculated values

Views

Simplify frequently required reports

Low Stock Products

```
SELECT ProductName, StockQty
FROM Product
WHERE StockQty < 10;
```

Units Sold per Product

```
SELECT ProductID,
  SUM(Quantity) AS UnitsSold
FROM SaleItem
GROUP BY ProductID;
```

Key Reports Generated

- Low Stock Products
- Stock Valuation
- Supplier Purchases
- Daily Sales
- Monthly Sales
- Product Returns
- Fast-Moving Products
- Profit Analysis

Implementation Progress & Database Prototype

Progress Checklist

- ✓ Requirements analysed
- ✓ Entities and relationships identified
- ✓ ER model designed
- ✓ Database normalized up to 3NF
- ✓ Tables and constraints implemented
- ✓ Sample data inserted
- ✓ SQL queries and reports tested
- ↻ *Front-end / database integration*
- ↻ *Final testing and validation*

Current Workflow



Planned / Remaining Work

- Java / Python front-end integration
- Product maintenance module
- Purchase and billing modules
- Customer search
- Returns and stock updates
- Management reports & final testing

The current prototype demonstrates a structured, normalized and constraint-driven retail database.